

```
import csv
import sqlite3
from pathlib import Path

class DatabaseConnector:
    """Populate the Walmart shipment SQLite database from CSV files."""

    def __init__(self, database_file):
        self.connection = sqlite3.connect(database_file)
        self.cursor = self.connection.cursor()

    def populate(self, spreadsheet_folder):
        """Read all three CSV files and populate the database."""
        folder = Path(spreadsheet_folder)

        file_0 = folder / "shipping_data_0.csv"
        file_1 = folder / "shipping_data_1.csv"
        file_2 = folder / "shipping_data_2.csv"

        with file_0.open("r", newline="", encoding="utf-8") as spreadsheet_0, \
            file_1.open("r", newline="", encoding="utf-8") as spreadsheet_1, \
            file_2.open("r", newline="", encoding="utf-8") as spreadsheet_2:

            reader_0 = csv.reader(spreadsheet_0)
            reader_1 = csv.reader(spreadsheet_1)
            reader_2 = csv.reader(spreadsheet_2)

            self.populate_first_shipping_data(reader_0)
            self.populate_second_shipping_data(reader_1, reader_2)

        self.connection.commit()

    def populate_first_shipping_data(self, csv_reader):
        """Load shipping_data_0.csv directly into the database."""
        next(csv_reader, None) # Skip header.

        for row in csv_reader:
            origin = row[0]
            destination = row[1]
            product_name = row[2]
            product_quantity = int(row[4])

            product_id = self.insert_product_if_needed(product_name)
            self.insert_shipment(
                product_id,
                product_quantity,
                origin,
                destination
            )

    def populate_second_shipping_data(self, products_reader, shipments_reader):
        """
        Combine shipping_data_1.csv and shipping_data_2.csv.

        shipping_data_1 has one product per row. Products with the same
        shipment identifier are counted so each shipment/product pair is
        inserted once with its total quantity.
        """
        shipment_info = {}

        # shipping_data_2 contains shipment origin/destination.
        next(shipments_reader, None) # Skip header.

        for row in shipments_reader:
            shipment_identifier = row[0]
            origin = row[1]
            destination = row[2]

            shipment_info[shipment_identifier] = {
                "origin": origin,
                "destination": destination,
                "products": {}
            }

        # shipping_data_1 contains one product per row.
        next(products_reader, None) # Skip header.

        for row in products_reader:
```

```
shipment_identifier = row[0]
product_name = row[1]

products = shipment_info[shipment_identifier]["products"]
products[product_name] = products.get(product_name, 0) + 1

# Insert one database row per product in each shipment.
for shipment in shipment_info.values():
    for product_name, product_quantity in shipment["products"].items():
        product_id = self.insert_product_if_needed(product_name)

        self.insert_shipment(
            product_id,
            product_quantity,
            shipment["origin"],
            shipment["destination"]
        )

def insert_product_if_needed(self, product_name):
    """Insert a product if it does not already exist and return its ID."""
    self.cursor.execute(
        """
        INSERT OR IGNORE INTO product (name)
        VALUES (?);
        """,
        (product_name,)
    )

    self.cursor.execute(
        """
        SELECT id
        FROM product
        WHERE name = ?;
        """,
        (product_name,)
    )

    return self.cursor.fetchone()[0]

def insert_shipment(self, product_id, product_quantity, origin, destination):
    """Insert a shipment/product record."""
    self.cursor.execute(
        """
        INSERT OR IGNORE INTO shipment
        (product_id, quantity, origin, destination)
        VALUES (?, ?, ?, ?);
        """,
        (product_id, product_quantity, origin, destination)
    )

def close(self):
    self.connection.close()

def main():
    database_file = "shipment_database.db"
    spreadsheet_folder = "data"

    database_connector = DatabaseConnector(database_file)

    try:
        database_connector.populate(spreadsheet_folder)
        print("Database population complete.")
    except Exception:
        database_connector.connection.rollback()
        raise
    finally:
        database_connector.close()

if __name__ == "__main__":
    main()
```

How the script satisfies the task

1. shipping_data_0.csv is read row-by-row and inserted directly.
2. shipping_data_2.csv is indexed by shipment identifier to provide origin and destination.
3. shipping_data_1.csv is grouped by shipment identifier and product name.
4. Each occurrence of a product represents one unit, so occurrences are counted to obtain quantity.
5. A product is inserted into the product table only when it does not already exist.
6. Each shipment/product pair is inserted into the shipment table with product_id, quantity, origin and destination.
7. Changes are committed once after successful processing; failures roll back the transaction.
8. sqlite3 and csv are used so the script requires no third-party packages.